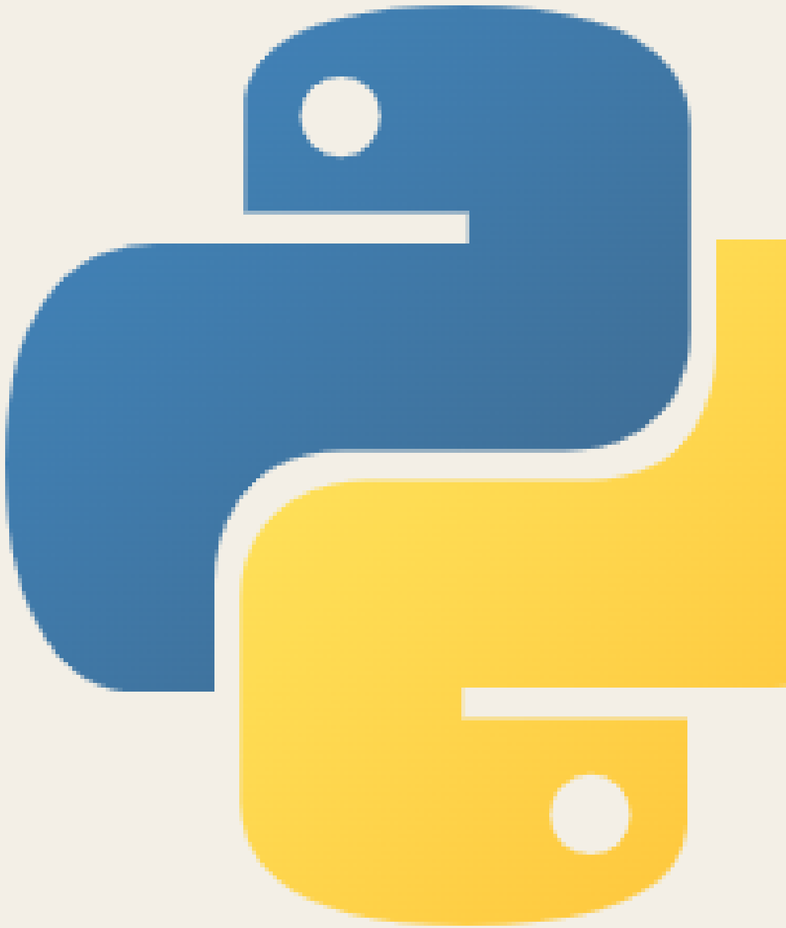
CFA INSTA
Institut National Supérieur des Technologies Avancées

TP Final · Python

```
>>> import socket, threading, tkinter
```

Chat client–serveur



Salah · serveur

Said · client 1

Nelson · client 2

Plan

01 — p. 03–05
Le projet
contexte · outils · installation

02 — p. 06–09
Le code
sockets · architecture · serveur · client

03 — p. 10–11
Le réseau
VPN NetBird · équipe

04 — p. 12–14
Améliorations
erreurs · Tkinter · recherche auto

05 — p. 15
Démo
en direct

06 — p. 16
Rendu
dépôt GitHub commun

Contexte

Chacun chez soi

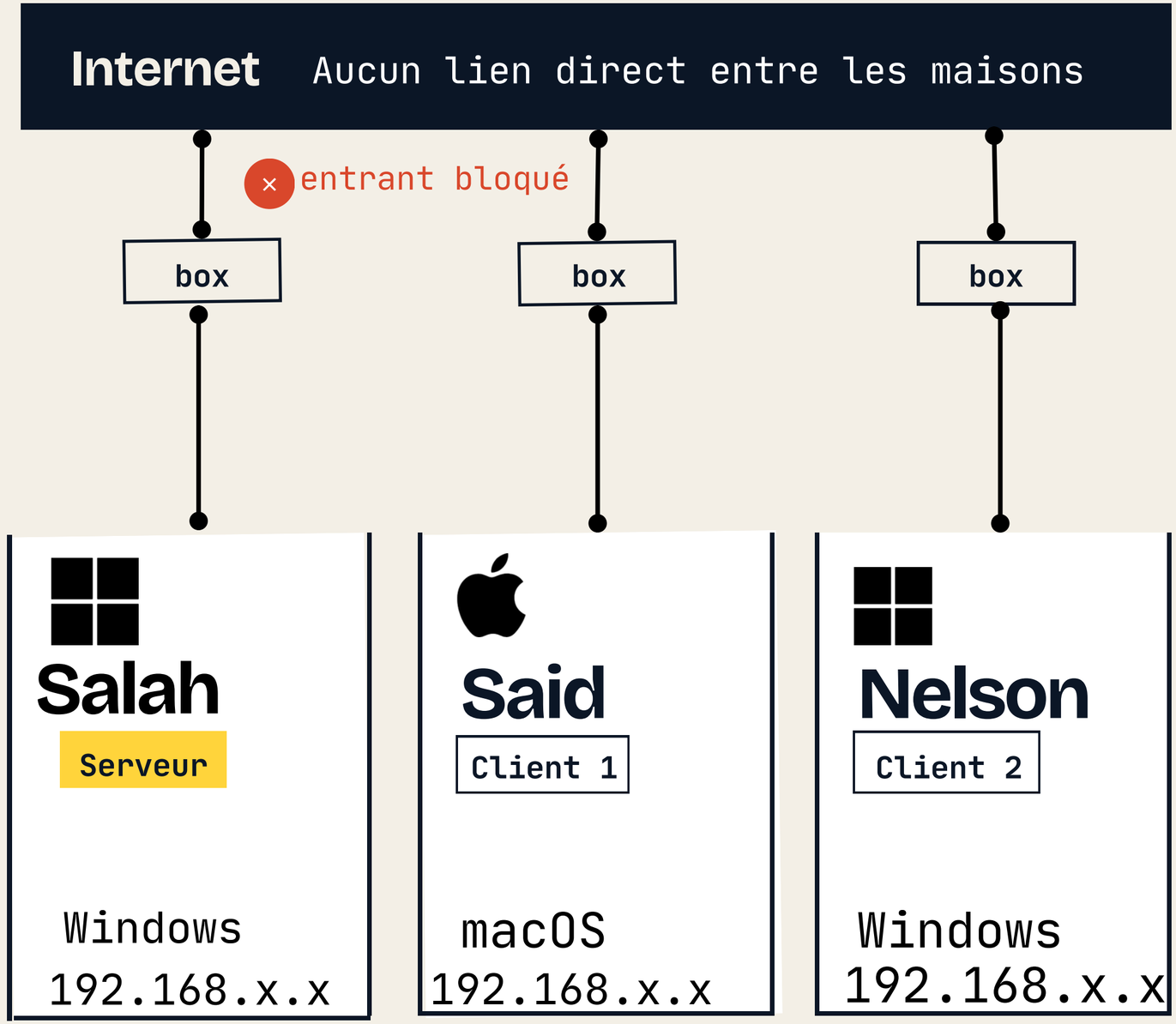
Le TP demande un serveur et deux clients. Nous travaillons à distance : chaque PC est derrière la box de sa maison.

Ce qui bloque

- × Les adresses 192.168 restent locales
- × La box refuse les connexions entrantes
- × Pas de Wi-Fi commun pour se trouver

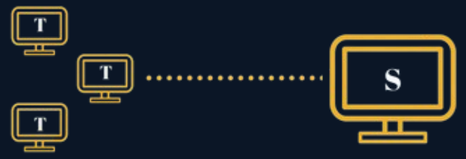
Notre solution

Un VPN NetBird pour communiquer ensemble



Nos outils

Python 3



Socket

Réseau TCP, un thread par client

Tkinter

```
import tkinter
```

Tkinter

La fenêtre du chat

NetBird

VPN basé sur WireGuard

VS Code

Ecrire et commenter le code

Git · GitHub

Un dépôt commun

Installation

Installer en 5 étapes

1 

Python 3

https://www.python.org/

```
>python --version
```

2 

NetBird

Création d'une Team + ajout de chaque user

```
>netbird up >netbird status
```

3 

Le code

Cloner le dépôt commun

```
> git clone https://github.com/NelsonCamara/tp-python-client-server
```

4 

Pare-feu

PC serveur uniquement

```
> New-NetFirewallRule -Direction Inbound -Protocol TCP -LocalPort 59001 -Action Allow
```

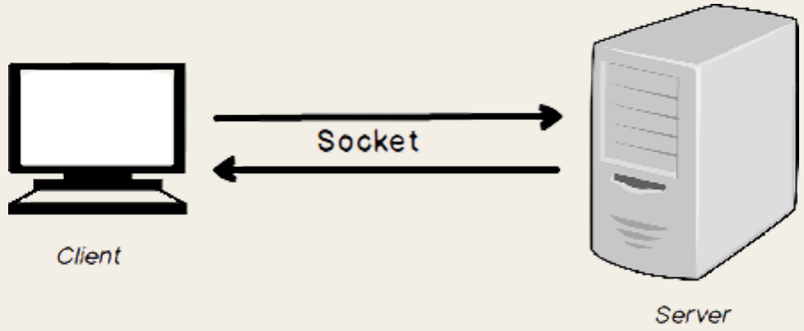
5 >_

Lancer

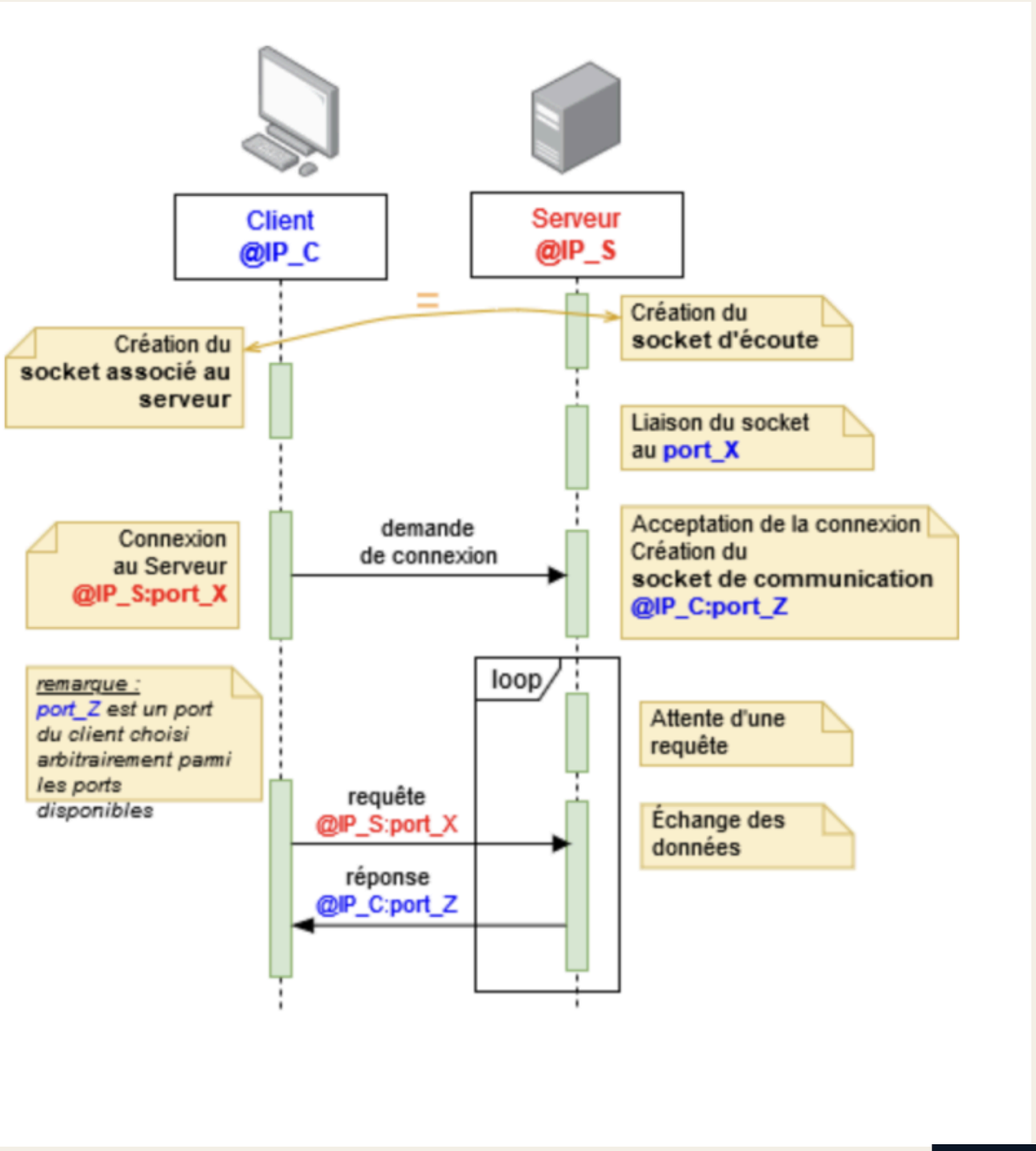
Serveur puis client

```
>python server.py >python client_gui.py
```

Comment marche une socket TCP

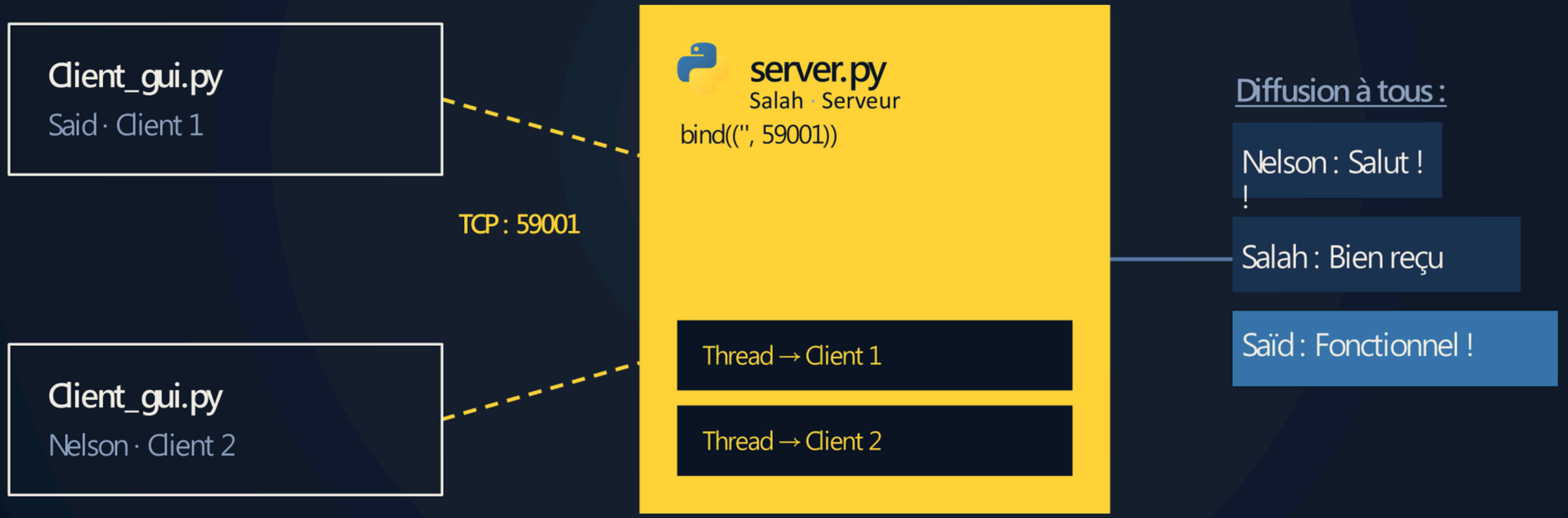


1. **socket()** crée la connexion
2. **bind()** lie le port 59001
3. **accept()** accueille un client
4. **`send / recv` en `boucle`**



Architecture

Un serveur, un thread par client



Code

Côté serveur

Server.py Ecoute et accepte

server.py

C: > Users > salah > Projects > tp_python_clientserver_chat > server.py > ...

```python
import socket # Permet de créer la connexion du serveur
import sys # Permet d'arrêter le programme s'il y a une erreur
import time # Permet d'attendre un petit moment
from clientthread import ClientListener # On récupère la classe qui gère un client

class Server(): # Cette classe représente le serveur

    def __init__(self, port): # Cette méthode prépare le serveur
        self.name= "Server Messagerie CFA INSTA - Said, Nelson, Salah" # Nom affiché pendant la recherche
        self.listener= socket.socket(socket.AF_INET, socket.SOCK_STREAM) # On crée la connexion du serveur
        self.listener.bind(('', port)) # On associe la socket au port
        self.listener.listen(1) # On met le serveur en attente
        print("Listening on port", port) # On affiche le port utilisé
        self.clients_sockets= [] # Liste des clients connectés

    def run(self): # Cette méthode contient la boucle principale
        while True: # Le serveur continue toujours d'attendre
            print("listening new customers") # On indique que le serveur attend un client
            try: # On essaie d'accepter une connexion
                (client_socket, client_adress) = self.listener.accept() # On récupère la socket et l'adresse du
            except socket.error: # Cette partie est utilisée en cas d'erreur réseau
                sys.exit("Cannot connect clients") # On arrête le programme
```

Clientthread.py Un thread par client

clientthread.py

C: > Users > salah > Projects > tp_python_clientserver_chat > clientthread.py > ...

```python
from socket import socket, error # Permet d'utiliser la connexion et les erreurs
import threading # Permet de gérer plusieurs clients en même temps
import re # Permet de trouver la commande USERNAME
import time # Permet d'attendre un petit moment


class ClientListener(threading.Thread): # Cette classe gère un client connecté

    def __init__(self, server, socket, address): # Cette méthode prépare le thread du client
        super(ClientListener, self).__init__() # On initialise la classe Thread
        self.server= server # On garde le serveur principal
        self.socket= socket # On garde la socket du client
        self.address= address # On garde l'adresse du client
        self.listening= True # Cette variable garde l'écoute active
        self.username= "No username" # Le pseudo sera reçu après la connexion


    def run(self): # Cette méthode se lance avec le thread
        while self.listening: # On continue tant que le client est connecté
            data= "" # On prépare une variable vide
            try: # On essaie de recevoir un message
                data = self.socket.recv(1024).decode('UTF-8') # On reçoit puis on transforme les données en texte
            except error: # Cette partie est utilisée si la réception échoue
                print("Unable to receive data") # On affiche une erreur
            self.handle_msg(data) # On traite le message reçu
            time.sleep(0.1) # On attend un petit moment
        print("Ending client thread for", self.address) # On affiche la fin du thread
```

Code

Côté client

Client.py Connexion + écoute

client.py

C: > Users > salah > Projects > tp_python_clientserver_chat > client.py

```python
import threading # On utilise un thread pour recevoir les messages
import socket # On utilise socket pour se connecter au serveur
import time # On utilise time pour attendre un petit moment
import re # On utilise re pour trouver la commande USERNAME

class Client(): # Cette classe représente un client

    def __init__(self, username, server, port, display=print): # Cette partie prépare le client
        self.socket= socket.socket(socket.AF_INET, socket.SOCK_STREAM) # On crée la connexion
        self.socket.connect((server, port)) # On se connecte au serveur
        self.username= username # On garde le pseudo
        self.display= display # On garde la façon d'afficher les messages
        self.send("USERNAME {0}".format(username)) # On envoie le pseudo au serveur
        self.listening= True # On indique que le client écoute

    def listener(self): # Cette fonction reçoit les messages
        while self.listening: # On continue tant que le client écoute
            data= "" # On prépare une variable vide
            try: # On essaie de recevoir un message
                data= self.socket.recv(1024).decode('UTF-8') # On reçoit le message
            except socket.error: # Cette partie est utilisée s'il y a une erreur
                print("Unable to receive data") # On affiche le problème
            self.handle_msg(data) # On traite le message reçu
            time.sleep(0.1) # On attend un petit moment

```

client_gui.py Fenêtre Tkinter

client_gui.py

C: > Users > salah > Projects > tp_python_clientserver_chat > client_gui.py

```python

import tkinter as tk # Pour créer notre fenêtre
import socket # Pour rechercher le serveur
from client import Client # On utilise le client du professeur

client = None # Aucun client connecté au début
ip = "100.66.75.9" # Adresse NetBird du serveur de Salah
port = 59001 # Port utilisé pour le chat

def afficher(texte): # Affiche un message dans le chat
    chat.insert(tk.END, texte + "\n") # Ajoute le message
    chat.see(tk.END) # Descend vers le dernier message

```

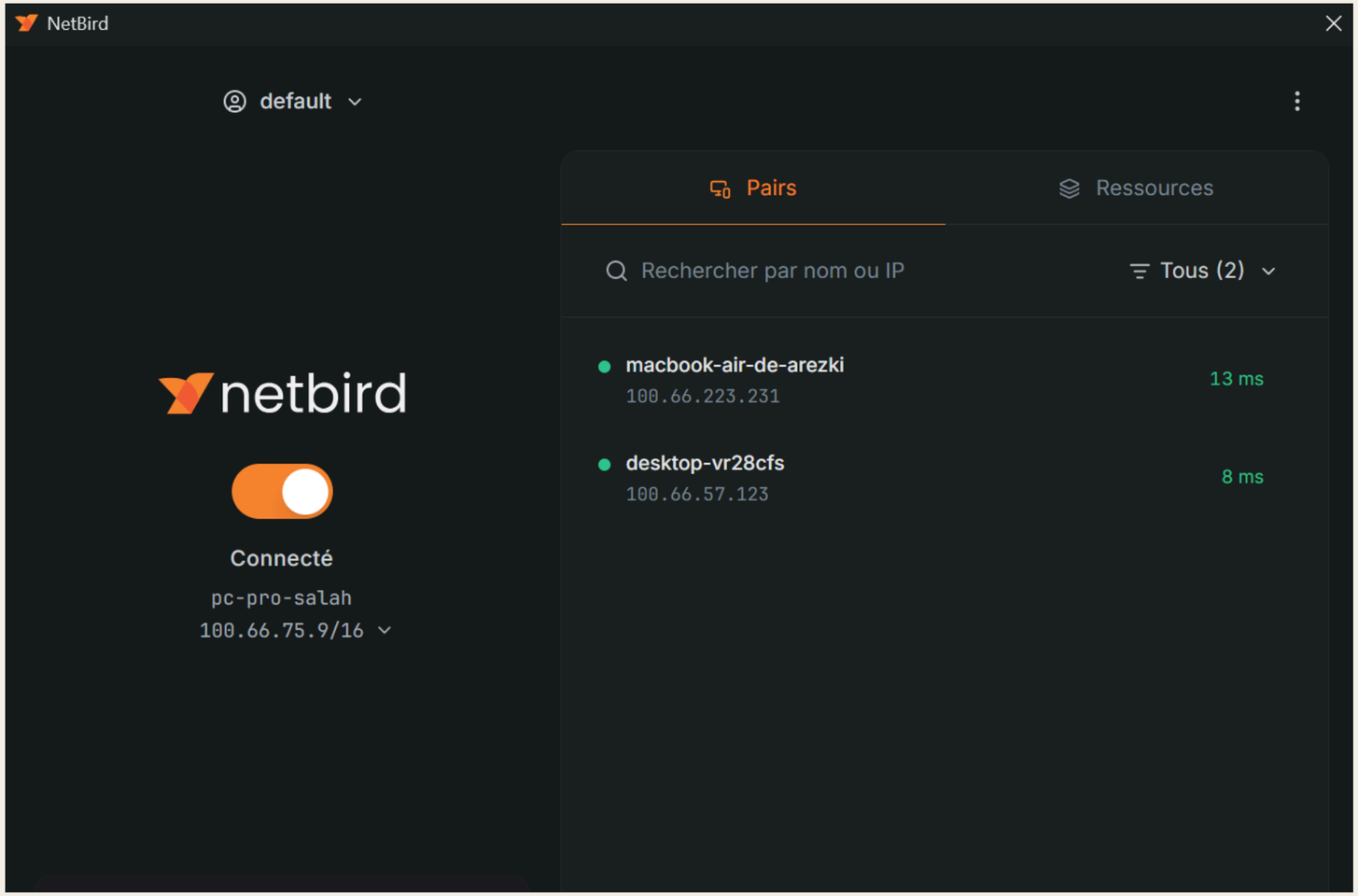
Réseau

NetBird, un VPN entre nos PC

Salah	`100.66.75.9`
Saïd	`100.66.223.231`
Nelson	`100.66.57.123`

pare-feu Windows

```
ALLOW TCP 59001
```

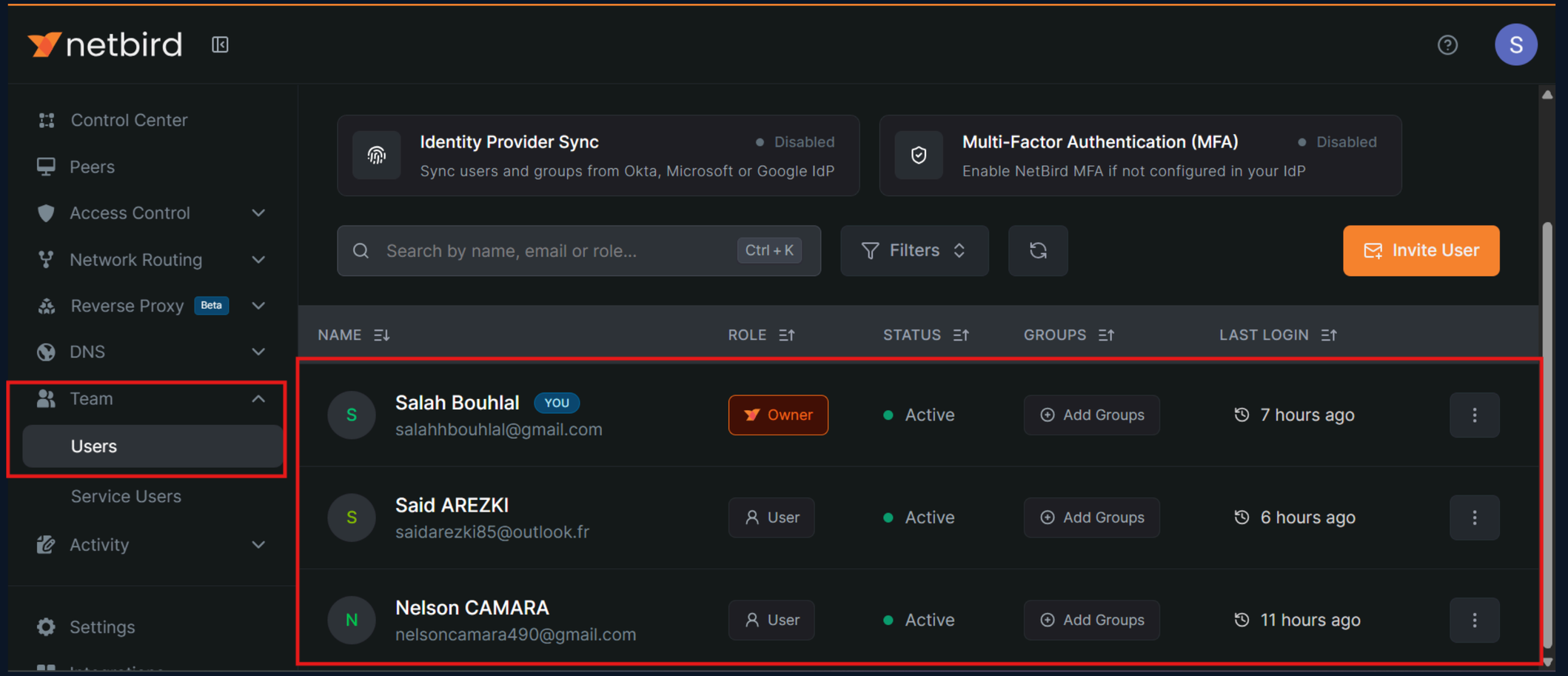


pc-salah + 2 pairs connectés

Réseau

Une équipe, un réseau privé

Salah · Owner | Said · client 1 | Nelson · client 2



Amélioration du code

Des messages d'erreur lisibles

AVANT

```
$ python client.py
Traceback (most recent call last):
File "client.py", line 14
sock.connect((host, port))
TimeoutError: [WinError 10060]
```

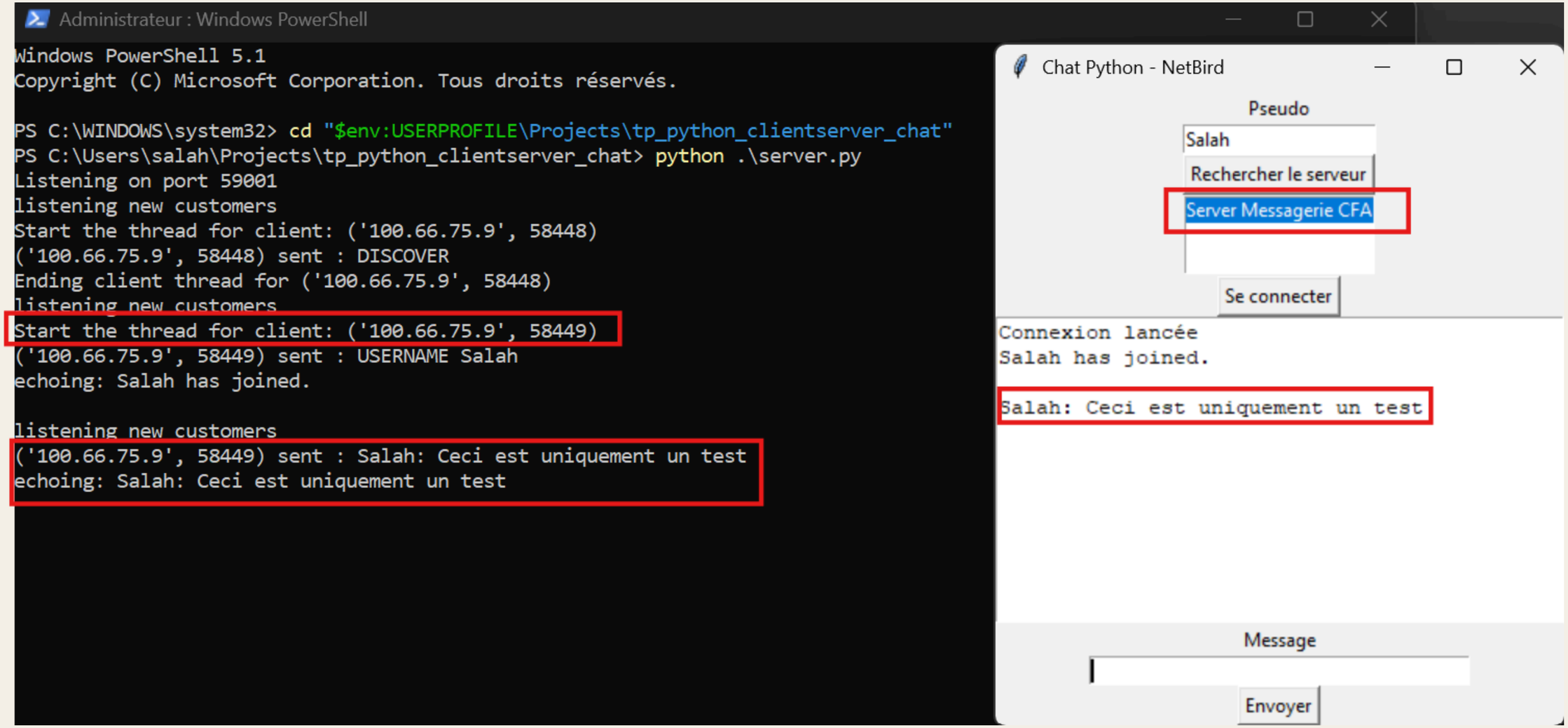
APRÈS

```
$ python client.py
Pseudo : Said
Adresse du serveur : 100.66.75.9
Port [59001] :
✗ Délai dépassé : le serveur
ne répond pas. Vérifiez NetBird.
```

Port par défaut | Saisies validées | /quit corrigé | Déconnexions gérées

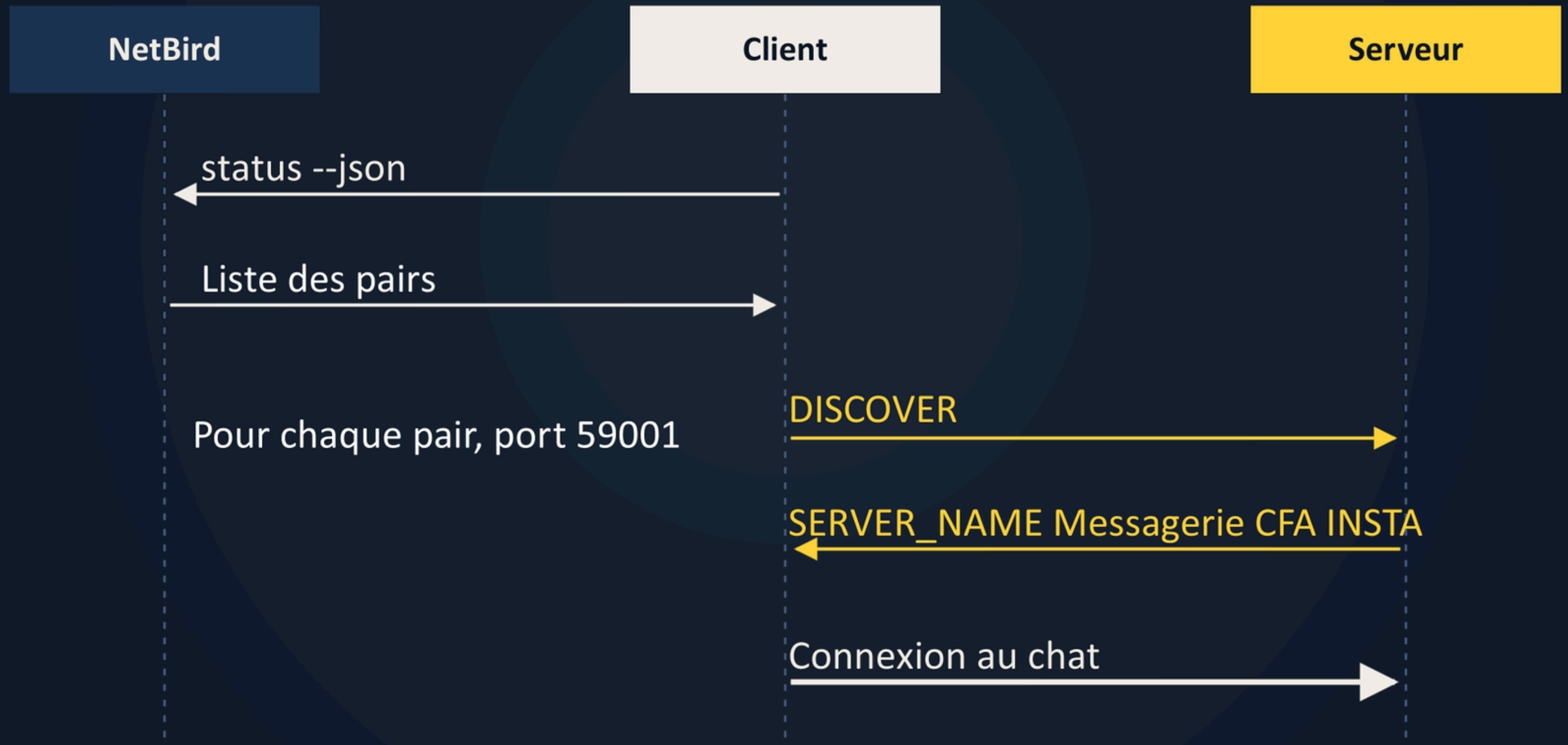
Rechercher → Se connecter → Envoyer

L'interface Tkinter



Option

Recherche automatique des serveurs



Démonstration

Démo

1 - Python server.py

```
Administrateur : Windows PowerShell
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. Tous droits réservés.

PS C:\WINDOWS\system32> cd "$env:USERPROFILE\Projects\tp_python_clientserver_chat"
PS C:\Users\salah\Projects\tp_python_clientserver_chat> python .\server.py
Listening on port 59001
listening new customers
```

2 - Python client_gui.py

```
Administrateur : Windows PowerShell
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. Tous droits réservés.

PS C:\WINDOWS\system32> cd "$env:USERPROFILE\Projects\tp_python_clientserver_chat"
PS C:\Users\salah\Projects\tp_python_clientserver_chat> python .\client_gui.py
```

3 - Discussion côté interface server

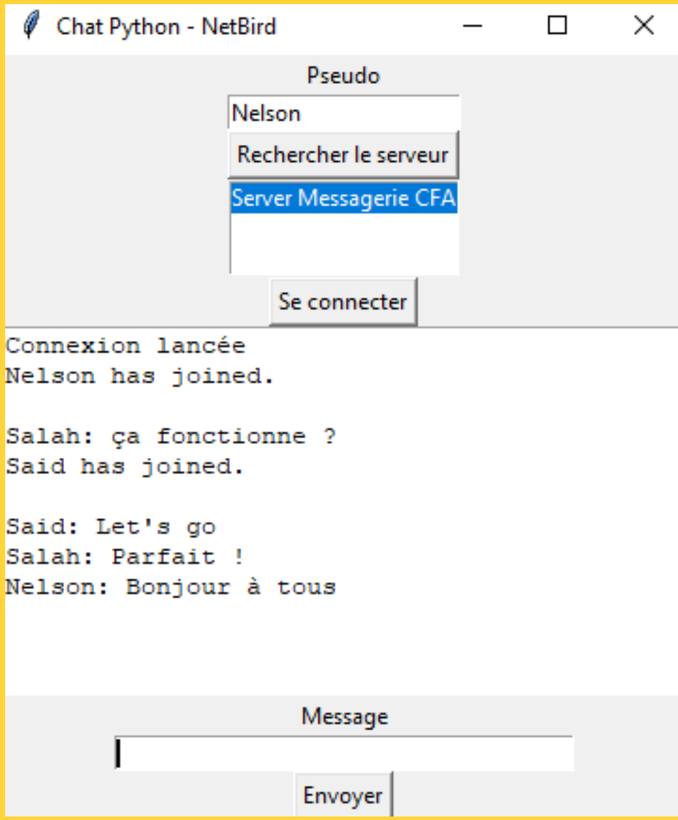
```
echoing: Salah has joined.

listening new customers
Start the thread for client: ('100.66.57.123', 62137)
('100.66.57.123', 62137) sent : DISCOVER
listening new customers
Ending client thread for ('100.66.57.123', 62137)
Start the thread for client: ('100.66.57.123', 62160)
('100.66.57.123', 62160) sent : USERNAME Nelson
echoing: Nelson has joined.

listening new customers
('100.66.75.9', 62270) sent : Salah: ça fonctionne ?
echoing: Salah: ça fonctionne ?
Start the thread for client: ('100.66.223.231', 61668)
('100.66.223.231', 61668) sent : DISCOVER
listening new customers
Ending client thread for ('100.66.223.231', 61668)
Start the thread for client: ('100.66.223.231', 61669)
('100.66.223.231', 61669) sent : USERNAME Said
echoing: Said has joined.

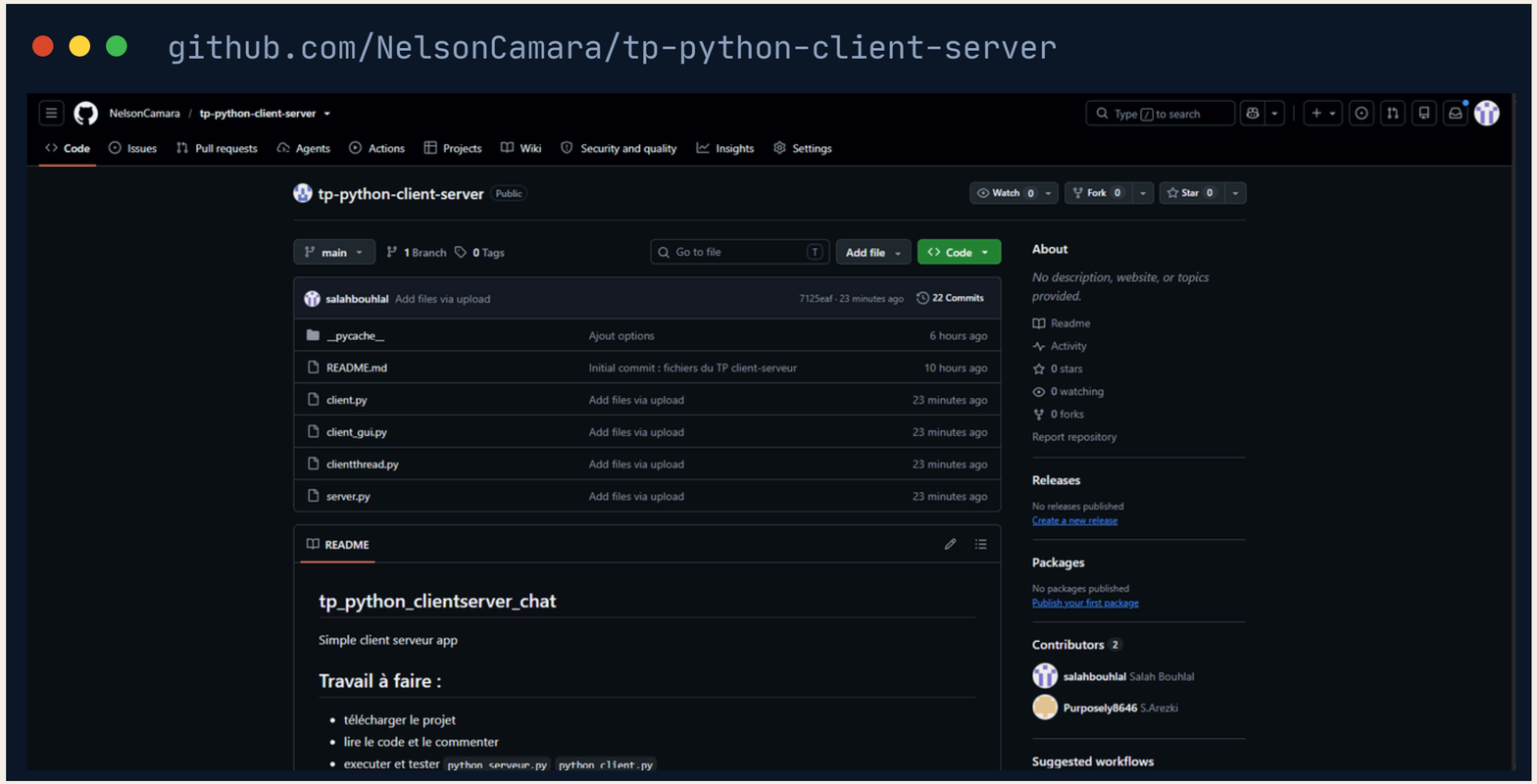
listening new customers
('100.66.223.231', 61669) sent : Said: Let's go
echoing: Said: Let's go
('100.66.75.9', 62270) sent : Salah: Parfait !
echoing: Salah: Parfait !
('100.66.57.123', 62160) sent : Nelson: Bonjour à tous
echoing: Nelson: Bonjour à tous
```

4 - Interface TKinter



Rendu

Un dépôt GitHub commun



GitHub

github.com/
NelsonCamara/
tp-python-client-server

Salah	Serveur
Saïd	Client 1
Nelson	Client 2

Merci.

URL envoyée à
f.cardenas@cfa-insta.fr